

SeatFindr – Smart Study Space Occupancy Monitoring System

COEN/ELEC 390 Computer and Electrical Engineering Product Design Project

Concordia University
January 2026 – April 2026

Document Prepared by: Sbaz Ali Khan

Table of Contents

<i>SeatFindr – Smart Study Space Occupancy Monitoring System.....</i>	<i>1</i>
<i>Project Overview</i>	<i>3</i>
<i>Problem and Motivation</i>	<i>4</i>
<i>Project Objectives.....</i>	<i>4</i>
<i>System Architecture</i>	<i>5</i>
<i>Hardware.....</i>	<i>6</i>
HLK-LD2450 mmWave Sensor	6
ESP32	6
<i>Occupancy Detection Logic.....</i>	<i>7</i>
<i>Firestore Realtime Database</i>	<i>10</i>
<i>Android Application</i>	<i>10</i>
Building and Floor Navigation	11
Smart Search.....	11
Room Availability	11
Occupancy Indicators	11
Sorting	12
Manual Refresh and Real-Time Updates	12
Room Recommendations	13
<i>Stakeholder Interviews and User Feedback.....</i>	<i>15</i>
<i>Development Methodology – Agile / Scrum.....</i>	<i>15</i>

Sprint 1	16
Sprint 2	16
Sprint 3	16
Computer Simulation.....	17
Testing and Validation.....	18
Sensor Testing.....	18
ESP32 and Firebase Testing.....	18
Android Testing.....	18
End-to-End Testing	19
Privacy and Ethical Considerations.....	19
Transparency	20
Data Use.....	20
Accessibility.....	20
Security.....	20
My Contributions	20
Skills and Technologies.....	21
Hardware and Embedded Systems.....	21
Programming	22
Development Tools	22
Cloud and Database	22
Mobile Development.....	22
Engineering Skills	22
Final Outcome	23

Project Overview

SeatFindr is a smart study-space occupancy monitoring system developed as part of the COEN/ELEC 390 Computer and Electrical Engineering Product Design Project at Concordia University.

The purpose of the project was to help students identify available study spaces before physically travelling between rooms, floors, or buildings. Students often spend unnecessary time searching for seating without knowing whether a study area is already occupied. SeatFindr addresses this problem by combining physical occupancy sensing, embedded processing, wireless communication, cloud-based data storage, and an Android mobile application.

The prototype uses an **HLK-LD2450 24 GHz mmWave radar sensor** connected to an **ESP32 microcontroller**. The sensor monitors a predefined seating area and detects the presence of seated occupants. The ESP32 processes the sensor data, determines the current occupancy level, and transmits the information over Wi-Fi to a **Firestore Realtime Database**.

A native Android application, developed in **Java and XML using Android Studio**, retrieves the occupancy information from Firestore and displays room availability to the user.

The complete system can therefore be summarized as:

HLK-LD2450 Sensor → ESP32 → Wi-Fi → Firestore Realtime Database → Android Application → User

The final prototype demonstrated a complete sensing-to-user pipeline in which a physical occupancy change could be detected by the sensor, processed by the ESP32, transmitted to Firestore, and displayed in the Android application.



Problem and Motivation

Finding an available place to study can be frustrating and time-consuming, particularly during busy periods such as exams or peak campus hours.

Students may walk to a study area only to discover that all seats are occupied. They may then need to move to another floor, room, or building and repeat the process. This wastes time and can make it more difficult for students to plan their study sessions efficiently.

The project was therefore designed around the idea of providing students with **near-real-time seat availability information before they arrive at a location**.

The main expected benefits were:

- reducing the time spent searching for study spaces.
- allowing students to make more informed decisions.
- improving the overall student experience.
- providing a foundation for smarter use of university study-space infrastructure.

The system was also intentionally designed to avoid the privacy concerns associated with camera-based monitoring. SeatFindr does not need to know who is using a seat; it only needs to know whether the seat or monitored area is occupied.

Project Objectives

The main objectives of the project were to:

- create a functional occupancy monitoring prototype.
- detect seated occupants within a defined study area.
- distinguish sustained occupancy from brief pass-through movement.
- detect up to three occupants using the HLK-LD2450 sensor.
- process occupancy information using an ESP32.
- transmit occupancy data wirelessly to a cloud database.
- display room and seat availability through an Android application.
- organize room information using a building, floor, and room hierarchy.
- provide a simple and intuitive interface for students.
- support room searching, sorting, navigation, and availability indicators.
- validate the complete sensor-to-application communication pipeline.
- incorporate stakeholder feedback into the design.
- consider privacy, accessibility, reliability, and scalability throughout development.

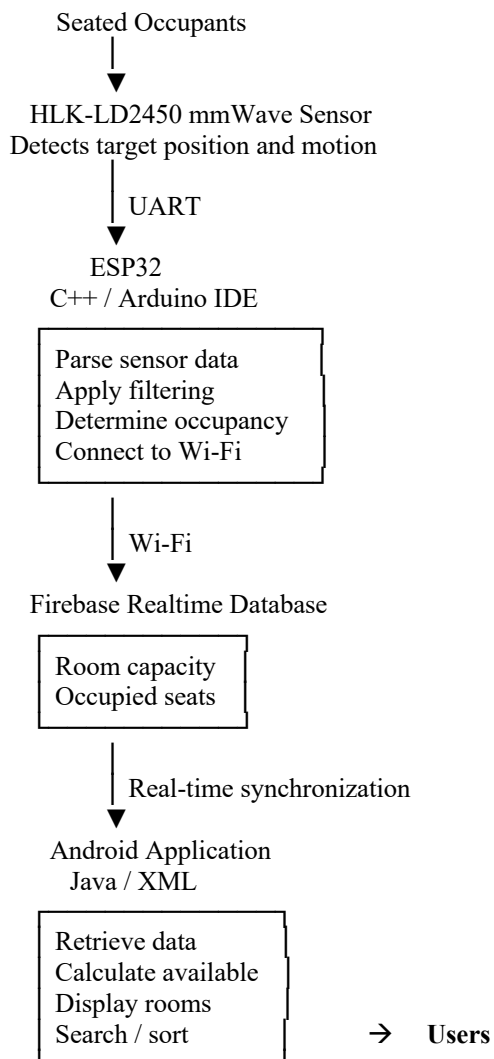
The prototype was intended to demonstrate the feasibility of the concept rather than represent a full university-wide deployment.

System Architecture

SeatFindr consists of four major technical layers:

1. **Sensing layer**
2. **Embedded processing layer**
3. **Cloud/database layer**
4. **Mobile application layer**

The architecture is shown conceptually below:



The Android application calculates availability using:

$$\text{Available Seats} = \text{Room Capacity} - \text{Occupied Seats}$$

This architecture allows the physical sensing hardware and the mobile application to operate independently while communicating through Firebase.

Hardware

The primary hardware components used in the prototype were:

- HLK-LD2450 mmWave radar sensor.
- ESP32 microcontroller.
- wiring and power connections.
- 3D-printed enclosure for the physical prototype.

HLK-LD2450 mmWave Sensor

The HLK-LD2450 is a 24 GHz millimeter-wave radar sensor capable of tracking up to three targets simultaneously.

The sensor was selected because it is capable of detecting small movements associated with human presence. This is particularly important for study environments because a student may remain relatively still while reading, typing, or working.

The sensor communicates with the ESP32 through **UART serial communication**.

The prototype wiring used:

- VCC → 5 V
- GND → GND
- TX → ESP32 GPIO14
- RX → ESP32 GPIO13

ESP32

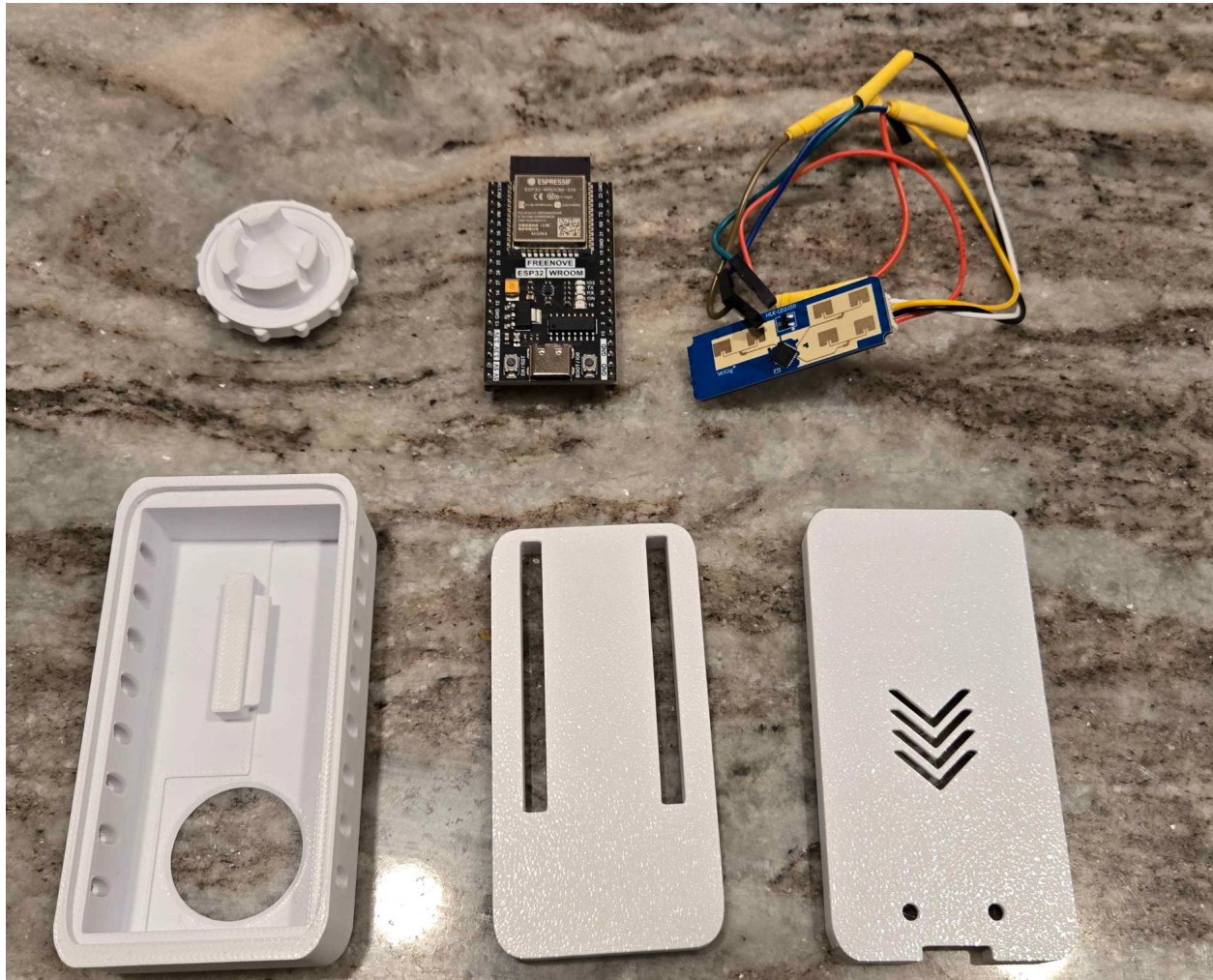
The ESP32 acts as the main processing and communication unit.

Its responsibilities include:

- receiving radar data from the LD2450.
- parsing target information.
- determining valid occupants.

- applying occupancy filtering logic.
- connecting to Wi-Fi.
- communicating with Firebase.
- updating occupancy information when changes occur.

The ESP32 firmware was developed in C++ using the **Arduino IDE**.



Occupancy Detection Logic

The prototype focused on a small, predefined seating area rather than attempting to monitor an entire classroom.

The final monitored area was designed around a **three-seat micro-zone**, which matched the practical capabilities of the LD2450 sensor.

The sensor continuously provides target information to the ESP32. The embedded software evaluates the detected targets and determines whether they fall inside the intended monitoring region.

The system was not intended to count every person who briefly passed through the sensor's field of view. Instead, the design focused on **sustained occupancy and micro-movement**.

A student sitting at a desk will normally create small movements over time, even if they are relatively still. In contrast, a person simply walking through the monitored area creates a temporary detection that should not necessarily represent an occupied seat.

Filtering and statistical polling were therefore used to make the occupancy value more stable and reduce false updates caused by temporary detections.

The system was tested under several conditions:

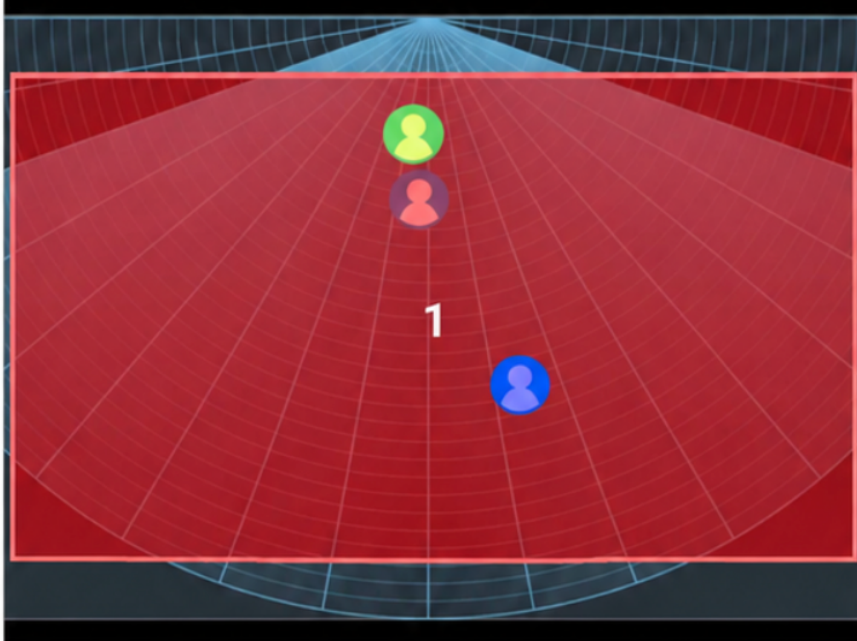
- one seated occupant.
- two seated occupants.
- three seated occupants.
- people walking past the monitored zone.
- people standing outside the intended area.
- different sensor distances.
- different sensor orientations.
- different seating positions.

The completed prototype successfully demonstrated detection of up to **three seated occupants** in the intended monitoring area.

< Back

HLK-LD2450_635C

Set



Target 1

Distance 2.58m
Angle -4.62°
Speed -0.08m/s°

Target 2

Distance 1.80m
Angle -8.89°
Speed 0.00m/s°

Target 3

Distance 4.94m
Angle 11.72°
Speed 0.00m/s°

Baud rate

Select Baud Rate >

Multipla
yer



Reboot Module

Restore Factory

MAC

340DE448635C

Version number(Click to perform
OTA upgrade)

2.14.25112412

Firestore Realtime Database

Firestore Realtime Database was used as the communication layer between the embedded hardware and the Android application.

The ESP32 acts as a data producer, while the Android application acts as a data consumer.

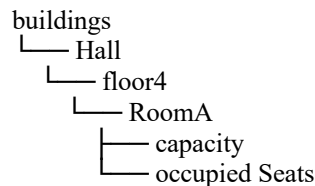
The communication flow is:

ESP32 → Firestore → Android Application

The ESP32 updates occupancy information when the detected seat count changes. Firestore then stores the current values so they can be retrieved by the Android application.

The database was designed using a hierarchical structure representing university geography.

A simplified example is:



The room capacity is a fixed value, while the occupied-seat count changes based on sensor information.

This approach also provides a foundation for future expansion because additional buildings, floors, and rooms can be represented using additional database paths.

Firestore Realtime Database was selected because it provides convenient synchronization between connected devices and mobile applications.

Android Application

The SeatFindr mobile application was developed using **Android Studio, Java, and XML**.

The Android application serves as the user-facing portion of the project. It converts raw occupancy information into a format that is useful and easy for students to understand.

The main interface allows the user to select a building and floor or use the search bar to navigate more quickly.

Major application features included:

Building and Floor Navigation

The application allows users to select buildings and floors using dropdown menus.

Buildings used during development included locations such as Hall, JMSB, and EV.

The available floor list updates based on the selected building.

Smart Search

A smart search feature was developed to make the application easier to use.

The search logic handles:

- uppercase and lowercase inputs.
- partial building names.
- building abbreviations.
- building and floor combinations.
- missing characters in supported cases.
- invalid floors.
- invalid building inputs.

For example, a user could enter a building and floor rather than selecting both manually.

The application validates the request before navigation and displays an appropriate error message if the requested floor does not exist.

Room Availability

The room activity displays:

- room name.
- total capacity.
- available seats.
- occupancy status.

Rooms can be displayed in a scrollable list so that floors with many study spaces can still be presented in a usable interface.

Occupancy Indicators

The application uses visual occupancy indicators to make room availability easier to interpret.

Conceptually:

- Green indicates good availability.
- Yellow indicates moderate occupancy.
- Red indicates high occupancy.

A status guide explains the meaning of the indicators.

Sorting

The application provides sorting options that allow rooms to be organized according to different criteria, including:

- availability percentage.
- number of available seats.
- occupancy level or seats occupied.
- room capacity.

This helps users quickly identify the most suitable study space.

Manual Refresh and Real-Time Updates

Firebase provides real-time synchronization, allowing the app to update as the underlying database values change.

A manual refresh function was also added to give the user an explicit way to request current information.

The application displays a manual-refresh timestamp so that this action is not confused with automatic Firebase updates.

Room Recommendations

The main interface was also extended to display highly available rooms for the selected building and floor, helping users make quicker decisions.

12:16

10

Seat Finder

Hall ▼

Floor 9 ▼

View Floor Rooms

Google Maps Address: 1455 De Maisonneuve
Blvd W, Montreal, QC H3G 1M8

Most Seats Available

H-905	 Available: 31 / 36
H-903	 Available: 22 / 42
H-901	 Available: 14 / 18

Viewing: Hall/floor9

Last manual refresh: Never

Highest Availability Percentage

Most Seats Available

Least Occupied

Highest Capacity

H-903

 Available: 22 / 42

Status Guide

-  Red — Crowded
-  Yellow — Partially Full
-  Green — Available

Stakeholder Interviews and User Feedback

The course required the project to consider stakeholder needs rather than designing the product only from the team's own assumptions.

The team prepared an interview script and collected feedback from Concordia students.

The interviews explored topics such as:

- current problems related to campus occupancy.
- areas where study-space availability is most important.
- expected system accuracy.
- desired update speed.
- privacy concerns.
- useful application features.
- trust in occupancy information.

The team also collected feedback from students who interacted with the prototype.

Feedback indicated that the application was generally easy to understand, but users suggested improvements such as:

- showing multiple rooms at the same time.
- improving navigation.
- clearly defining what "real-time" means.

These comments directly influenced later development.

For example, the application was expanded to support multiple room entries and improved user navigation. The team also defined its real-time testing target as approximately **1–2 seconds between a Firebase value change and the corresponding Android interface update.**

Development Methodology – Agile / Scrum

The project was developed using an **Agile/Scrum-style workflow.**

Development activities were organized using:

- Product Backlog.
- User Stories.
- Story Points.
- Sprint Backlogs.

- Sprint Goals.
- Task Estimates.
- Definition of Done.
- Sprint testing.
- demonstrations and reviews.

The product evolved through multiple iterations rather than being built as one final implementation.

Sprint 1

Sprint 1 focused on creating the basic sensing-to-application pipeline.

Major work included:

- connecting the LD2450 sensor to the ESP32;
- reading and parsing sensor data;
- creating basic occupancy logic;
- connecting the ESP32 to Wi-Fi;
- setting up Firebase Realtime Database;
- sending occupancy values to Firebase;
- creating the initial Android application;
- connecting the Android app to Firebase;
- developing the initial sensor simulation.

The Sprint 1 velocity was **21 story points**.

Sprint 2

Sprint 2 focused on improving detection capability and application usability.

Major improvements included:

- detecting up to three occupants;
- optimizing sensor placement and detection range;
- validating multi-user detection;
- adding a scrollable room interface;
- sorting rooms by availability;
- adding an occupancy legend;
- displaying building addresses;
- implementing smart search;
- validating building and floor inputs.

The Sprint 2 velocity was approximately **23 story points**.

Sprint 3

Sprint 3 focused on physical integration and further Android application improvements.

Major completed work included:

- mounting and casing integration;
- manual refresh functionality;
- additional sorting options;
- room recommendations;
- improved occupancy indicators;
- interface refinements;
- administration-related features within the team project;
- further system testing.

The Sprint 3 velocity was approximately **25 story points**.

Later backlog items considered future expansion such as multi-sensor deployment, broader multi-room support, and support for additional institutions. These were treated as scalability directions rather than features fully demonstrated in the final physical prototype.

Computer Simulation

A computer simulation was developed to support sensor-selection and placement decisions.

The simulation represented the monitored seating environment as a simplified 2D coordinate grid.

Students were represented as points or small motion regions inside the seating zone, while the sensor's field of view was represented geometrically.

The purpose of the simulation was to investigate:

- sensor field-of-view coverage;
- suitable sensor placement;
- blind spots;
- effective monitoring boundaries;
- detection outside the intended seating area;
- possible overlap between multiple sensors;
- future scaling challenges.

One of the most important results of the simulation was the decision to reduce the initial prototype scope from a large study area to a smaller **three-seat micro-zone**.

This made the prototype more realistic and better aligned with the practical capabilities of the LD2450.

The simulation also confirmed that sensor placement is an important design variable. Poor placement can produce blind spots or detect people outside the intended area, while correct placement improves useful coverage and reliability.

The simulation therefore helped connect theoretical coverage analysis with practical hardware placement and testing.

Testing and Validation

Testing was performed at multiple levels.

Sensor Testing

The team tested whether the sensor could:

- detect one seated occupant;
- detect two seated occupants;
- detect three seated occupants;
- ignore people walking outside or near the monitored area.

These tests passed under the tested prototype conditions.

ESP32 and Firebase Testing

The system was tested to verify that:

- the ESP32 connected to Wi-Fi;
- occupancy data could be transmitted to Firebase;
- the database reflected occupancy changes;
- updated values were stored correctly.

These tests also passed.

Android Testing

The Android application was tested for:

- Firebase connectivity;
- room-capacity retrieval;
- occupied-seat retrieval;

- available-seat calculation;
- building and floor navigation;
- invalid search handling;
- room sorting;
- scrolling;
- building-address display;
- occupancy indicators;
- manual refresh;
- Firebase updates.

End-to-End Testing

The most important validation was the complete system-level test:

Physical Occupancy Change → LD2450 → ESP32 → Firebase → Android Application

The prototype successfully demonstrated this complete sequence.

The Android application updated within approximately **1–2 seconds after a Firebase database change** during testing.

Privacy and Ethical Considerations

Privacy was one of the main design considerations from the beginning of the project.

Any system that monitors activity in shared spaces can raise concerns, even when it does not identify individuals.

SeatFindr was therefore designed to collect as little information as necessary.

The prototype does not require:

- cameras;
- photographs;
- video;
- audio;
- facial recognition;
- names;
- student IDs.

The system is only interested in anonymous occupancy information.

For example, the application needs to know that a room contains three occupants; it does not need to know who those occupants are.

The project also considered several broader ethical issues.

Transparency

Students should understand what installed sensors are doing.

In a real deployment, signage or application information could explain that the sensors only measure occupancy and do not record images or identities.

Data Use

Occupancy information should remain focused on its intended purpose: helping users locate available spaces.

It should not later be repurposed for individual tracking or disciplinary monitoring.

Accessibility

The application was designed to communicate occupancy using more than only raw numerical information.

Visual indicators and interface improvements were considered to make the application easier to interpret.

Accessibility improvements for users with color-vision deficiencies were also explored during development.

Security

Although the system does not store personally identifying information, access to Firebase and administrative controls should still be protected in a real deployment.

My Contributions

This project was completed as a team project, and the work described throughout this document represents the overall system developed by the group.

My personal contributions focused primarily on the Android application, Firebase integration, embedded hardware integration, and end-to-end testing.

My contributions included:

- Android application UI development;
- development and refinement of the Main Activity;
- development and refinement of the room-occupancy interface;
- smart search implementation;
- input normalization and validation;
- building and floor navigation logic;
- room sorting and filtering functionality;
- navigation and toolbar features;
- help functionality;
- manual refresh functionality;
- manual-refresh timestamp feedback;
- Firebase Realtime Database setup;
- development of the initial Firebase JSON/database hierarchy;
- contributing to Firebase-to-Android application integration;
- initial ESP32 and LD2450 hardware connection using Arduino IDE;
- sensor testing and calibration;
- hardware/software integration;
- testing of the complete sensor-to-application pipeline.

I also contributed to the physical prototype enclosure by sourcing a suitable existing 3D enclosure model and arranging its 3D printing for the prototype. I did not design the enclosure CAD model from scratch.

My largest contribution was in the **Android application and user-facing functionality**, while I also gained practical experience in embedded systems, sensor integration, cloud databases, and system-level integration.

Skills and Technologies

The project provided experience across several technical areas.

Hardware and Embedded Systems

- ESP32
- HLK-LD2450 mmWave radar sensor
- UART serial communication
- sensor integration
- embedded-system testing
- hardware/software integration

Programming

- Java
- C++
- XML

Development Tools

- Android Studio
- Arduino IDE

Cloud and Database

- Firebase
- Firebase Realtime Database
- JSON hierarchical data structures
- real-time application synchronization

Mobile Development

- Android application development
- UI development
- navigation logic
- user-input validation
- sorting and filtering
- Firebase Android integration

Engineering Skills

- IoT system integration
 - system architecture
 - sensor calibration
 - debugging
 - system testing
 - requirements analysis
 - stakeholder feedback
 - Agile/Scrum development
 - privacy-conscious engineering design
-

Final Outcome

SeatFindr successfully demonstrated a complete smart occupancy-monitoring prototype combining:

physical sensing → embedded processing → wireless communication → cloud database → mobile visualization

The HLK-LD2450 sensor detected occupants within a monitored study area, the ESP32 processed the sensor data and transmitted occupancy values through Wi-Fi, Firebase stored and synchronized the information, and the Android application displayed room availability to the user.

The prototype demonstrated that these technologies could be integrated into a functional IoT system for study-space monitoring.

The project also showed the importance of considering more than technical functionality alone. Stakeholder feedback, sensor placement, privacy, accessibility, interface design, testing, and scalability all influenced the final design.

Future development could extend the concept through additional sensors, larger monitored areas, more rooms and buildings, improved sensor-zone aggregation, stronger access control, and broader smart-campus deployment.

Overall, the project provided practical experience in **embedded systems, IoT integration, Firebase, Android development, sensor technology, Agile/Scrum development, and end-to-end engineering system design.**